

lab6

```
1  function lab6()
2  % =====
3  % File: lab6.m
4  % Course: Queueing Systems - Lab 6
5  % Topic: Time-Series Forecasting
6  %
7  % Basic part only - no validation set:
8  %   1. Try 5 Linear Regression models
9  %   2. Try 5 Filter models
10 %   3. Select the best model in each category using median CRPS
11 %   4. Plot only:
12 %       - best Linear Regression model
13 %       - best Filter model
14 %       - comparison between the two selected models
15 %
16 % Required file:
17 %   dataset.mat must be in the same directory as this .m file.
18 % =====
19
20 close all;
21 clc;
22
23 % Octave needs the statistics package for norminv, normcdf, normpdf.
24 if exist('pkg', 'builtin') || exist('pkg', 'file')
25     try
26         pkg load statistics;
27     catch
28         warning('Could not load Octave statistics package. Make sure norminv, normcdf and
29 normpdf are available.');
```

end

```
30 end
31
32 % =====
33 % 1. Load dataset
34 % =====
35
36 dataset_file = 'dataset.mat';
37
38 data = load(dataset_file);
39
40 if ~isfield(data, 'values')
41     error('dataset.mat does not contain a variable named values.');
```

end

```
42
43
44 y = double(data.values(:));
45
46 n = length(y);
47 nob = 1200; % first 50 days = 1200 hours
48 alpha = 0.05; % 95% prediction interval
49 z = norminv(1 - alpha / 2); % approximately 1.96
```

```

50
51 if n <= nob
52     error('The dataset must contain more than 1200 observations.');
```

end

```

54
55 nforecast = n - nob;
56 t_all = (1:n)';
57 forecast_idx = (nob + 1:n)';
58
59 fprintf('\n=====\\n');
60 fprintf('Lab 6 - Basic Forecasting with Model Selection\\n');
61 fprintf('=====\\n');
62 fprintf('Loaded dataset: %s\\n', dataset_file);
63 fprintf('Number of observations: %d\\n', n);
64 fprintf('Training observations: %d\\n', nob);
65 fprintf('Forecast observations: %d\\n', nforecast);
66 fprintf('Forecast horizon: %d to %d\\n', nob + 1, n);
67 fprintf('Confidence level: 95%\\n');
```

68

```

69 % =====
70 % 2. Evaluate 5 Linear Regression models
71 % =====
72
73 reg_results(1) = evaluate_regression_model(y, nob, alpha, ...
74     'LR1: constant + daily 24', ...
75     0, [24]);
76
77 reg_results(2) = evaluate_regression_model(y, nob, alpha, ...
78     'LR2: linear trend + daily 24', ...
79     1, [24]);
80
81 reg_results(3) = evaluate_regression_model(y, nob, alpha, ...
82     'LR3: linear trend + daily 24 + weekly 168', ...
83     1, [24, 168]);
84
85 reg_results(4) = evaluate_regression_model(y, nob, alpha, ...
86     'LR4: quadratic trend + daily 24 + weekly 168', ...
87     2, [24, 168]);
88
89 reg_results(5) = evaluate_regression_model(y, nob, alpha, ...
90     'LR5: linear trend + daily 24 + half-daily 12 + weekly 168', ...
91     1, [24, 12, 168]);
92
93 best_reg_idx = find_best_by_crps(reg_results);
94 best_reg = reg_results(best_reg_idx);
95
96 % =====
97 % 3. Evaluate 5 Filter models
98 % =====
99
100 filter_results(1) = evaluate_filter_model(y, nob, alpha, ...
101     'F1: Delta_24', ...
102     make_diff_filter([24]));
103
```

```

104 filter_results(2) = evaluate_filter_model(y, nob, alpha, ...
105     'F2: Delta_1 Delta_24', ...
106     make_diff_filter([1, 24]));
107
108 filter_results(3) = evaluate_filter_model(y, nob, alpha, ...
109     'F3: Delta_168', ...
110     make_diff_filter([168]));
111
112 filter_results(4) = evaluate_filter_model(y, nob, alpha, ...
113     'F4: Delta_1 Delta_168', ...
114     make_diff_filter([1, 168]));
115
116 filter_results(5) = evaluate_filter_model(y, nob, alpha, ...
117     'F5: Delta_24 Delta_168', ...
118     make_diff_filter([24, 168]));
119
120 best_filter_idx = find_best_by_crps(filter_results);
121 best_filter = filter_results(best_filter_idx);
122
123 % =====
124 % 4. Print comparison tables
125 % =====
126
127 fprintf('\n===== \n');
128 fprintf('Linear Regression Candidate Models \n');
129 fprintf('===== \n');
130 fprintf('%-60s %8s %12s %12s %12s %12s \n', ...
131     'Model', 'Params', 'Train MSE', 'MAE', 'RMSE', 'Med CRPS');
132
133 for i = 1:length(reg_results)
134     fprintf('%-60s %8d %12.3f %12.3f %12.3f %12.3f \n', ...
135         reg_results(i).name, ...
136         reg_results(i).num_params, ...
137         reg_results(i).train_mse, ...
138         reg_results(i).mae, ...
139         reg_results(i).rmse, ...
140         reg_results(i).median_crps);
141 end
142
143 fprintf('\nBest Linear Regression model by median CRPS: \n');
144 fprintf(' %s \n', best_reg.name);
145
146 fprintf('\n===== \n');
147 fprintf('Filter Candidate Models \n');
148 fprintf('===== \n');
149 fprintf('%-35s %8s %12s %12s %12s %12s \n', ...
150     'Model', 'Order', 'Res Std', 'MAE', 'RMSE', 'Med CRPS');
151
152 for i = 1:length(filter_results)
153     fprintf('%-35s %8d %12.3f %12.3f %12.3f %12.3f \n', ...
154         filter_results(i).name, ...
155         filter_results(i).order, ...
156         filter_results(i).residual_std, ...
157         filter_results(i).mae, ...

```

```

158     filter_results(i).rmse, ...
159     filter_results(i).median_crps);
160 end
161
162 fprintf('\nBest Filter model by median CRPS:\n');
163 fprintf(' %s\n', best_filter.name);
164
165 fprintf('\n===== \n');
166 fprintf('Final Selected Models Comparison\n');
167 fprintf('===== \n');
168 fprintf('%-35s %12s %12s %12s %12s\n', ...
169     'Selected model', 'MAE', 'RMSE', 'Coverage', 'Med CRPS');
170
171 fprintf('%-35s %12.3f %12.3f %11.2f%% %12.3f\n', ...
172     best_reg.name, ...
173     best_reg.mae, ...
174     best_reg.rmse, ...
175     100 * best_reg.coverage, ...
176     best_reg.median_crps);
177
178 fprintf('%-35s %12.3f %12.3f %11.2f%% %12.3f\n', ...
179     best_filter.name, ...
180     best_filter.mae, ...
181     best_filter.rmse, ...
182     100 * best_filter.coverage, ...
183     best_filter.median_crps);
184
185 if best_reg.median_crps < best_filter.median_crps
186     fprintf('\nOverall, the selected Linear Regression model has better median
CRPS.\n');
187 elseif best_filter.median_crps < best_reg.median_crps
188     fprintf('\nOverall, the selected Filter model has better median CRPS.\n');
189 else
190     fprintf('\nThe selected models have equal median CRPS.\n');
191 end
192
193 write_results_csv('lab6_model_comparison_results.csv', reg_results, filter_results);
194
195 % =====
196 % 5. Plots only for the selected best models
197 % =====
198
199 save_figures = true;
200 npast = min(168, nob - 1);
201 past_idx = (nob - npast:nob)';
202
203 % Figure 1: Dataset and train-test split
204 figure(1);
205 plot(t_all(1:nob), y(1:nob), 'k');
206 hold on;
207 plot(t_all(forecast_idx), y(forecast_idx), 'o-b');
208 yl = ylim;
209 line([nob nob], yl, 'linestyle', '--', 'color', 'k');
210 grid on;

```

```

211 legend('Training data', 'Forecast/test data', 'Train-test split', ...
212         'location', 'northwest');
213 title('Dataset and train-test split');
214 xlabel('Hour');
215 ylabel('GPU resource requests');
216 hold off;
217
218 % Figure 2: Best regression fit on training data
219 figure(2);
220 plot(t_all(1:nob), y(1:nob), 'k');
221 hold on;
222 plot(t_all(1:nob), best_reg.yhat(1:nob), 'r');
223 grid on;
224 legend('Training data', 'Best linear regression fit', ...
225         'location', 'northwest');
226 title(['Best linear regression fit: ', best_reg.name]);
227 xlabel('Hour');
228 ylabel('GPU resource requests');
229 hold off;
230
231 % Figure 3: Best regression forecast
232 figure(3);
233 plot(t_all(past_idx), y(past_idx), 'k');
234 hold on;
235 plot(t_all(forecast_idx), y(forecast_idx), 'o-b');
236 plot(t_all(forecast_idx), best_reg.yhat(forecast_idx), 'r');
237 plot(t_all(forecast_idx), best_reg.upper, 'r-.');
238 plot(t_all(forecast_idx), best_reg.lower, 'r-.');
239 grid on;
240 legend('Past observed data', 'True future values', ...
241         'Best regression forecast', 'Upper 95% PI', 'Lower 95% PI', ...
242         'location', 'northwest');
243 title(['Best linear regression forecast: ', best_reg.name]);
244 xlabel('Hour');
245 ylabel('GPU resource requests');
246 hold off;
247
248 % Figure 4: Best filter residual diagnostics
249 figure(4);
250 res_x = (best_filter.order + 1:nob)';
251 plot(res_x, best_filter.filtered_residuals_centered, 'b');
252 hold on;
253 line([res_x(1) res_x(end)], [0 0], 'linestyle', '--', 'color', 'k');
254 grid on;
255 legend('Centered filtered residuals', 'Zero mean reference', ...
256         'location', 'northwest');
257 title(['Best filter residual diagnostics: ', best_filter.name]);
258 xlabel('Hour');
259 ylabel('Filtered residual');
260 hold off;
261
262 % Figure 5: Best filter forecast
263 figure(5);
264 plot(t_all(past_idx), y(past_idx), 'k');

```

```

265 hold on;
266 plot(t_all(forecast_idx), y(forecast_idx), 'o-b');
267 plot(t_all(forecast_idx), best_filter.yhat(forecast_idx), 'r');
268 plot(t_all(forecast_idx), best_filter.upper, 'r-.');
269 plot(t_all(forecast_idx), best_filter.lower, 'r-.');
270 grid on;
271 legend('Past observed data', 'True future values', ...
272        'Best filter forecast', 'Upper 95% PI', 'Lower 95% PI', ...
273        'location', 'northwest');
274 title(['Best filter forecast: ', best_filter.name]);
275 xlabel('Hour');
276 ylabel('GPU resource requests');
277 hold off;
278
279 % Figure 6: Point forecast comparison
280 figure(6);
281 plot(t_all(forecast_idx), y(forecast_idx), 'o-k');
282 hold on;
283 plot(t_all(forecast_idx), best_reg.yhat(forecast_idx), 'r');
284 plot(t_all(forecast_idx), best_filter.yhat(forecast_idx), 'b');
285 grid on;
286 legend('True future values', 'Selected regression forecast', 'Selected filter
forecast', ...
287        'location', 'northwest');
288 title('Point forecast comparison of selected models');
289 xlabel('Hour');
290 ylabel('GPU resource requests');
291 hold off;
292
293 % Figure 7: Absolute error comparison
294 figure(7);
295 plot(t_all(forecast_idx), abs(best_reg.errors), 'r');
296 hold on;
297 plot(t_all(forecast_idx), abs(best_filter.errors), 'b');
298 grid on;
299 legend('Selected regression absolute error', 'Selected filter absolute error', ...
300        'location', 'northwest');
301 title('Absolute forecast error comparison of selected models');
302 xlabel('Hour');
303 ylabel('Absolute error');
304 hold off;
305
306 if save_figures
307     print(1, 'lab6_fig1_dataset_train_test.png', '-dpng');
308     print(2, 'lab6_fig2_best_regression_fit.png', '-dpng');
309     print(3, 'lab6_fig3_best_regression_forecast.png', '-dpng');
310     print(4, 'lab6_fig4_best_filter_residuals.png', '-dpng');
311     print(5, 'lab6_fig5_best_filter_forecast.png', '-dpng');
312     print(6, 'lab6_fig6_selected_point_forecast_comparison.png', '-dpng');
313     print(7, 'lab6_fig7_selected_absolute_error_comparison.png', '-dpng');
314
315     fprintf('\nSaved figures as PNG files in the current directory.\n');
316 end
317

```

```

318 fprintf('\nSaved model comparison metrics to lab6_model_comparison_results.csv.\n');
319 fprintf('Execution completed successfully.\n');
320
321 end
322
323 % =====
324 % Subfunctions
325 % =====
326
327 function result = evaluate_regression_model(y, nob, alpha, model_name, degree,
periods)
328
329     n = length(y);
330     z = norminv(1 - alpha / 2);
331     forecast_idx = (nob + 1:n)';
332
333     X_all = build_regression_matrix(n, degree, periods);
334     X_train = X_all(1:nob, :);
335
336     p = size(X_train, 2);
337
338     b = X_train \ y(1:nob);
339     yhat = X_all * b;
340
341     residuals = y(1:nob) - X_train * b;
342     train_mse = sum(residuals .^ 2) / nob;
343
344     denom = max(nob - p, 1);
345     sigma2 = sum(residuals .^ 2) / denom;
346     sigma = sqrt(sigma2);
347
348     G = pinv(X_train' * X_train);
349
350     nforecast = n - nob;
351     cint = zeros(nforecast, 1);
352
353     for i = 1:nforecast
354         x0 = X_all(nob + i, :);
355         g = x0 * G * x0';
356         cint(i) = z * sigma * sqrt(1 + g);
357     end
358
359     lower = yhat(forecast_idx) - cint;
360     upper = yhat(forecast_idx) + cint;
361
362     [mae, rmse, coverage, median_crps, crps, errors] = ...
363         compute_forecast_metrics(y(forecast_idx), yhat(forecast_idx), lower, upper, z);
364
365     result.name = model_name;
366     result.type = 'regression';
367     result.degree = degree;
368     result.periods = periods;
369     result.num_params = p;
370     result.coefficients = b;

```

```

371     result.yhat = yhat;
372     result.lower = lower;
373     result.upper = upper;
374     result.cint = cint;
375     result.residuals = residuals;
376     result.train_mse = train_mse;
377     result.mae = mae;
378     result.rmse = rmse;
379     result.coverage = coverage;
380     result.median_crps = median_crps;
381     result.crps = crps;
382     result.errors = errors;
383
384 end
385
386 function X = build_regression_matrix(n, degree, periods)
387
388     t = (1:n)';
389     tau = t / 24; % time measured in days for numerical scaling
390
391     p = degree + 1 + 2 * length(periods);
392     X = zeros(n, p);
393
394     col = 1;
395
396     for k = 0:degree
397         X(:, col) = tau .^ k;
398         col = col + 1;
399     end
400
401     for j = 1:length(periods)
402         T = periods(j);
403
404         X(:, col) = cos(2 * pi * t / T);
405         col = col + 1;
406
407         X(:, col) = sin(2 * pi * t / T);
408         col = col + 1;
409     end
410
411 end
412
413 function result = evaluate_filter_model(y, nob, alpha, model_name, filter_coeffs)
414
415     n = length(y);
416     z = norminv(1 - alpha / 2);
417     forecast_idx = (nob + 1:n)';
418     nforecast = n - nob;
419
420     a = filter_coeffs(:)';
421     q = length(a) - 1;
422
423     if a(1) == 0
424         error('The first filter coefficient must be nonzero.');
```



```

425     end
426
427     if q >= nob
428         error('Filter order is too high for the available training set.');
```

429 end

```

430
431     raw_filtered = apply_filter_to_training(y(1:nob), a);
432
433     residual_mean = mean(raw_filtered);
434     centered_residuals = raw_filtered - residual_mean;
435     residual_std = std(centered_residuals);
436
437     yhat = y;
438     cint = zeros(nforecast, 1);
439
440     imp = [1; zeros(nforecast - 1, 1)];
441     inv_h = filter(1, a, imp);
442
443     for k = nob + 1:n
444         horizon = k - nob;
445
446         prediction = residual_mean;
447
448         for j = 1:q
449             prediction = prediction - a(j + 1) * yhat(k - j);
450         end
451
452         prediction = prediction / a(1);
453         yhat(k) = prediction;
454
455         sigma_h = residual_std * sqrt(sum(inv_h(1:horizon) .^ 2));
456         cint(horizon) = z * sigma_h;
457     end
458
459     lower = yhat(forecast_idx) - cint;
460     upper = yhat(forecast_idx) + cint;
461
462     [mae, rmse, coverage, median_crps, crps, errors] = ...
463         compute_forecast_metrics(y(forecast_idx), yhat(forecast_idx), lower, upper, z);
464
465     result.name = model_name;
466     result.type = 'filter';
467     result.coefficients = a;
468     result.order = q;
469     result.yhat = yhat;
470     result.lower = lower;
471     result.upper = upper;
472     result.cint = cint;
473     result.filtered_residuals_raw = raw_filtered;
474     result.filtered_residuals_centered = centered_residuals;
475     result.residual_mean = residual_mean;
476     result.residual_std = residual_std;
477     result.mae = mae;
478     result.rmse = rmse;

```

```

479     result.coverage = coverage;
480     result.median_crps = median_crps;
481     result.crps = crps;
482     result.errors = errors;
483
484 end
485
486 function res = apply_filter_to_training(y_train, a)
487
488     q = length(a) - 1;
489     n = length(y_train);
490     m = n - q;
491
492     res = zeros(m, 1);
493
494     for i = 1:m
495         t = q + i;
496         value = 0;
497
498         for j = 0:q
499             value = value + a(j + 1) * y_train(t - j);
500         end
501
502         res(i) = value;
503     end
504
505 end
506
507 function a = make_diff_filter(lags)
508
509     a = 1;
510
511     for i = 1:length(lags)
512         L = lags(i);
513
514         if L < 1
515             error('All differencing lags must be positive integers.');

```

```

533
534     mae = mean(abs(errors));
535     rmse = sqrt(mean(errors .^ 2));
536     coverage = mean((y_true >= lower) & (y_true <= upper));
537
538     sigma = (upper - mu) / z;
539     sigma(sigma <= 0) = eps;
540
541     crps = zeros(length(y_true), 1);
542
543     for i = 1:length(y_true)
544         a = (y_true(i) - mu(i)) / sigma(i);
545         crps(i) = sigma(i) * ...
546             (a * (2 * normcdf(a) - 1) + 2 * normpdf(a) - 1 / sqrt(pi));
547     end
548
549     median_crps = median(crps);
550
551 end
552
553 function idx = find_best_by_crps(results)
554
555     values = zeros(length(results), 1);
556
557     for i = 1:length(results)
558         values(i) = results(i).median_crps;
559     end
560
561     [~, idx] = min(values);
562
563 end
564
565 function write_results_csv(filename, reg_results, filter_results)
566
567     fid = fopen(filename, 'w');
568
569     if fid < 0
570         warning('Could not write results CSV file.');
```

```
586     for i = 1:length(filter_results)
587         fprintf(fid, 'filter,"%s",%.10f,%.10f,%.10f,%.10f,%.10f\n', ...
588             filter_results(i).name, ...
589             filter_results(i).residual_std, ...
590             filter_results(i).mae, ...
591             filter_results(i).rmse, ...
592             filter_results(i).coverage, ...
593             filter_results(i).median_crps);
594     end
595
596     fclose(fid);
597
598 end
```